

(c) 方策改善

価値関数を大きくするような行動を取るようの方策 π を更新する。具体的には、すべての s について以下を計算する。

$$\pi(s) := \arg \max_a \hat{Q}^\pi(s, a) = \arg \max_a \phi(s, a)' \mathbf{w}$$

上記のアルゴリズムを実装してみよ。

横軸に学習エピソード数、縦軸に各エピソードで得た累積報酬 (勝敗) をプロットした図を学習曲線と言う。学習曲線を書いてみよ。学習曲線を書くためには、方策改善を行う毎に、その方策を評価するためのシミュレーションを行う必要がある。

2 カートボールの安定化制御

強化学習によりカートボールの安定化制御を行う。

2.1 カートボールの設定

台車 (カート) へ適切に力を加えることで、カートの上に立てた棒 (ボール) が転倒しないようにバランスを取る。カートの質量 $M = 1.0(kg)$ 、ボールの質量 $m = 0.1(kg)$ 、ボールの長さ $2l = 1(m)$ 、重力加速度 $g = 9.8(m/sec^2)$ 、カートに加える力 $a(N)$ 、カートの摩擦係数 $\mu_c = 0.0005$ 、ボールの摩擦係数 $\mu_p = 0.000002$ とする。カートボールのダイナミクスは以下のように与えられる。

$$\begin{aligned} \ddot{y} &= \frac{g \sin(y) + \cos(y)(\mu_c \operatorname{sgn}(\dot{x}) - a - ml\dot{y}^2 \sin(y))/(M + m) - \mu_p \dot{y}/(ml)}{l(4/3 - m \cos^2(y)/(M + m))} \\ \ddot{x} &= \frac{a + ml(\dot{y}^2 \sin(y) - \ddot{y} \cos(y)) - \mu_c \operatorname{sgn}(\dot{x})}{M + m} \end{aligned}$$

ここで $x, \dot{x}, \ddot{x}$ はそれぞれ、カートの位置、速度、加速度を表す。また、 $y, \dot{y}, \ddot{y}$ はそれぞれ、垂直方向を $0(rad)$ としたときのボール角度、角速度、角加速度を表す。 $\operatorname{sgn}(\cdot)$ は、符号を返す関数である。時定数を $\tau = 1/60(sec^{-1})$ とし、 $1/60$ 秒おきに

$$x := x + \tau \dot{x}, \quad \dot{x} := \dot{x} + \tau \ddot{x}, \quad y := y + \tau \dot{y}, \quad \dot{y} := \dot{y} + \tau \ddot{y} \quad (5)$$

のように状態を遷移させる。 $a = \pm 20(N)$ を超える場合には、環境は超過分を無視して行動を実行する。

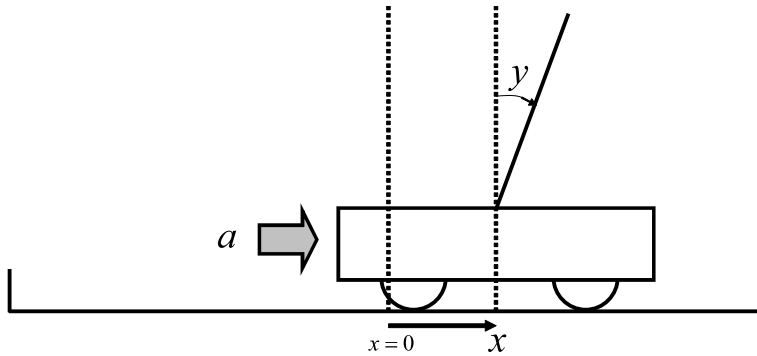


Figure 1: カートボール

2.2 アルゴリズム

以下に“方策勾配法”のアルゴリズムを示す。

1. 確率的方策の関数形の決定

ここでは、方策を正規分布で与える。方策パラメータは4次元ベクトル θ とスカラー η である。

$$\pi(a|s; \theta, \eta) = \mathcal{N}(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(a - \mu)^2}{2\sigma^2}\right)$$

ここで、

$$\begin{aligned} \mu &= \theta' C s, \\ C &= \text{diag}(1/2.4, 1/2, 180/(12\pi), 1/1.5), \\ \sigma &= 0.1 + \frac{1}{1 + \exp(\eta)} \end{aligned}$$

とする。 $s = [x, \dot{x}, y, \dot{y}]'$ とした。 C は s の各次元の値を $[-1, 1]$ の範囲に正規化するために用いている。 $\text{diag}(\cdot)$ は対角行列を表す。

2. パラメータの初期化

方策パラメータをランダムに初期化する。たとえば、 $\theta_1, \dots, \theta_4$ をそれぞれ、 $[-5, 5]$ の範囲でランダムに選択し、 η を $[-1, 1]$ の範囲でランダムに選択する。割引係数を $\gamma = 0.95$ とする。学習係数 α を適当に決める。たとえば $\alpha = 0.1$ とする。

3. 学習エピソード

以下の学習エピソードを繰り返す。 $|x| > 2.4(m)$ 、 $|\dot{x}| > 2(m/sec)$ 、 $|y| > 12\pi/180(rad)$ 、 $|\dot{y}| > 1.5(rad/sec)$ のいずれかを満たす場合、エピソードを終了し、新しいエピソードを始める。この課題では、ある程度良い制御が得られるまで10万エピソード以上を要する。

(a) 初期状態の選択: $s \sim \mathcal{N}(\mathbf{0}, \Sigma)$ 。ここで、 $\Sigma = \text{diag}(0.01, 0.01, 0.01, 0.01)$ 。

(b) 行動の選択: $a \sim \pi(a|s; \theta, \eta)$ 。

(c) 即時報酬の計算: $r = -s' Q s - a R a$ 。ここで、 $Q = \text{diag}(1.25, 1, 12, 0.25)$ 、 $R = 0.01$ とする。

(d) 方策勾配の逐次更新 (各エピソードの始めに、 $z = 0, \delta = 0, t = 0$ の初期化を行う)

$$\begin{aligned} z &:= z + [\nabla_{\theta} \ln \pi(a|s; \theta, \eta)', \nabla_{\eta} \ln \pi(a|s; \theta, \eta)]' \\ \delta &:= \delta + z r \gamma^t \\ t &:= t + 1 \end{aligned}$$

(e) 式 (5) に従い、状態遷移を行う。

4. 方策改善

各エピソードの終了後に、方策改善を行うかどうか判定する。前のエピソードまでに得られた方策勾配の推定値 Δ_{past} と今回のエピソードまでに得られた方策勾配の推定値 Δ の角度 $\angle(\Delta_{past}, \Delta)$ がある基準 ϵ (たとえば $\epsilon = 3/1000$) よりも小さければ、方策勾配の推定値が収束したとみなしてパラメータの更新を行う。具体的な計算式はたとえば以下ようになる。

$$\Delta_{past} := \Delta, \quad \Delta := \frac{n-1}{n} \Delta + \frac{1}{n} \delta, \quad n := n + 1$$

方策改善を行う場合、以下の計算を追加する。

$$[\theta', \eta]' := [\theta', \eta]' + \alpha \Delta, \quad n := 1$$

角度 $\angle(\Delta_{past}, \Delta)$ は余弦定理などを用いて計算する。

上記のアルゴリズムを実装せよ。また、学習曲線、ボールを安定に制御できた時間などを書いてみよ。

3 強化学習による迷路の探索

迷路の自動探索を行うエージェントを TD 学習法で訓練する。

3.1 アルゴリズム

以下に “epsilon-greedy TD(0)” のアルゴリズムを示す。ここで “epsilon-greedy” は方策の名前、“TD(0)” は価値関数の学習法を示している。

1. パラメータ ϵ, γ, α を適当に決める。
2. 価値関数を 0 に初期化する。すなわち、迷路の各座標 (x, y) について、 $V(x, y) = 0$ とする。
3. 以下の学習エピソードを繰り返す。ここで停止条件は特にないので、パフォーマンス（例えば 1 エピソードにおけるゴールまでの回数）を表示などして、十分に良くなったなら停止させるなどが考えられる。
 - (a) エージェント位置の初期化
エージェントを初期位置 (S) におく。
 - (b) 行動の選択
次の時刻でのエージェントの位置を以下の epsilon-greedy 方策で決める。
 - 確率 $1 - \epsilon$ で、 $\arg \max_{x', y'} (r(x, y, x', y') + \gamma V(x', y'))$ とする。ここで x', y' は、現在の位置から移動できるマスである。壁方向へは移動できないものとする。すなわち移動できるマスの中で価値関数が最大になるマスへ移動する。
 - 確率 ϵ でランダム（例えば移動できるマスが 4 つある場合は、0.25 ずつ）とする。
 - (c) 瞬時報酬の計算
エージェントの新しい位置 (x', y') がゴール (G) ならば $r = 1$ 、そうでなければ $r = 0$ とする。
 - (d) 評価関数の更新

$$V(x, y) := V(x, y) + \alpha(r + \gamma V(x', y') - V(x, y))$$

- (e) (x', y') がゴールでなければ $(x, y) \leftarrow (x', y')$ として、(b) に戻る。すなわちエージェントを移動する。

図 2 の迷路に対して上記のアルゴリズムを実装し、学習曲線を書かせてみよ。うまく行かせるためにはパラメータ γ, α を適切に定める必要がある。 γ は 1 以下で 1 に近い値が良い（例えば $\gamma \approx 0.95$ ）。 α は小さいほど学習が安定するが、あまりに小さいと学習の進行が遅い。

4 DQN の実装

近年、強化学習の近似 TD 法において、柔軟な近似性能をもつ深層学習を適用した深層強化学習の研究が盛んに行われている。そこで深層強化学習のなかでも基本的なアルゴリズムである Deep Q-Network (DQN) アルゴリズムの実装を行い、強化学習手法を高次元な問題へ適用する。

4.1 問題設定：数字の昇順ソート

N 桁の数字を昇順にソートする問題を考える。 i 桁目の数字を $x_i \in \{0, \dots, 9\}$ として、以下に各時刻 t における変数を定義する：

観測 s_t

N 桁の数字 $x_0 x_1 \dots x_{N-1}$

行動 a_t

$i+1 (i=0, \dots, N-2)$ 番目の隣接する 2 つの数字 x_i と x_{i+1} を交換する操作。確率 1 で a_t の結果に対応する観測 s_{t+1} を得るものとする。

報酬 r_t

数列を昇順にソートできた時点で報酬 1 を、それ以外の場合は、小さな負の報酬を与える。具体的には、最悪の場合に必要な行動数である $N(N-1)/2$ で -1 を割った値とする。

4.2 アルゴリズム

上記の問題設定における状態の総数は 10^N となり、例えば表形式の Q 学習ではその実装が技術的に困難になる。そこで、Q 関数をニューラルネットワークによる非線形関数近似手法を用いたアプローチを考える。まず Q 関数 $Q(s, a)$ がパラメータ θ を用いて表されていて、近似された関数を $Q_\theta(s, a)$ で表されるとする。また $Q_\theta(s, a)$ と同一のネットワークアーキテクチャをもつ関数 $\hat{Q}_{\bar{\theta}}(s, a)$ (ターゲットネットワーク) を用意する。以下に DQN 法のアルゴリズムを示す。

1. θ 、キュー P を初期化、 $\bar{\theta} = \theta$, $i = 0$
2. 各エピソードについて以下を繰り返す
 - (a) 状態の初期化： $t = 0$ 、 s_t の観測
 - (b) 以下を繰り返す
 - i. epsilon-greedy 方策に従い行動 a_t を選択
確率 $1 - \epsilon$ で $a_t = \arg \max_a Q_\theta(s_t, a)$ 、確率 ϵ でランダム
 - ii. 行動 a_t の実行し、次の状態 s_{t+1} と即時報酬 r_t を観測する
 - iii. サンプル $b_t = (s_t, a_t, r_t, s_{t+1})$ をキュー P にプッシュ
 - iv. $A < i$ の時 Q を θ に関して更新
 - v. $i \bmod B = 0$ の時、 $\bar{\theta} = \theta$
 - vi. s_{t+1} が終端状態でない時、 $i \leftarrow i + 1$ 、 $s_t \leftarrow s_{t+1}$ 、 $t \leftarrow t + 1$

4.3 学習の安定化

DQN 法では、上記アルゴリズムの 2 (b) iv での Q 関数の学習に関して種々の工夫を導入することにより学習の安定化に成功した。

4.3.1 Fixed Target Q-network

DQN アルゴリズムでは全ての s_t, a_t に対する値 $Q(s_t, a_t)$ に関して、

$$y_t = \begin{cases} r_t & s_{t+1} \text{ が終端状態} \\ r_t + \max_{a'} \hat{Q}_{\bar{\theta}}(s_{t+1}, a') & \text{otherwise} \end{cases} \quad (1)$$

に平均的に近づくように学習が行われる。ここで、用いられている Q 関数の値はターゲットネットワーク $\hat{Q}_{\bar{\theta}}$ の出力であることに注意されたい。主たるネットワークである $Q_{\theta}(s, a)$ の古いパラメータ $\bar{\theta}$ を用いたターゲットネットワークの出力を用いることが正則化として働き、学習が安定する。このような条件のもとで $Q_{\theta}(s, a)$ の学習を行う。具体的には、各時刻 t_i における誤差関数として二乗誤差

$$L_{\theta} = \frac{1}{2} (y_{t_i} - Q_{\theta}(s_{t_i}, a_{t_i}))^2 \quad (2)$$

が定義される。ネットワーク $Q_{\theta}(s, a)$ の学習はこの L_{θ} の θ に関する勾配

$$\nabla_{\theta} L_{\theta} = -(\text{target} - Q_{\theta}(s_{t_i}, a_{t_i})) \nabla_{\theta} Q_{\theta}(s_{t_i}, a_{t_i}) \quad (3)$$

を用いた確率的勾配降下法によって最適化される。

4.3.2 Experience Replay

これは従来の Q 学習のように 1 ステップごとにその時刻の内容 (experience) を学習するのではなく、キューに各時刻の内容を保存しておき、キューから内容をランダムに取り出して (replay)、ニューラルネットワークを学習する工夫である。すなわち

$$L_{\theta} = \mathbb{E}_{b_{t_i} \sim U(P)} \left[\frac{1}{2} (y_{t_i} - Q_{\theta}(s_{t_i}, a_{t_i}))^2 \right] \quad (4)$$

ここで $b_{t_i} \sim U(P)$ はキュー P からランダムにサンプルを取り出す操作を表す。各ステップごとにその時刻の内容を学習すると、時間的に相関が高いサンプルばかりになり、バイアスが生じる。ランダムに選ばれたサンプル集合によるミニバッチ学習を行うことで、バイアスの発生を抑制することができる。

4.3.3 Huber 関数

DQN アルゴリズムでは、実際には誤差関数として二乗誤差ではなく Huber 関数を使用する工夫がなされている。Huber 関数は誤差が -1 から 1 の間は二乗誤差の値となり、-1 より小さいか 1 より大きいときには誤差の絶対値をとる関数である。誤差が大きい場合に二乗誤差を使用すると、誤差関数の出力が大きくなりすぎて学習が安定しづらくなるため、Huber 関数によって安定化を図る。

4.4 課題

近年ニューラルネットワークの最適化に関し広く普及している、python による機械学習ライブラリーである Tensorflow や Pytorch を用いて、上記の DQN 法のアルゴリズムを実装し、数字の昇順ソートタスクへ

と適用し、学習曲線をプロットせよ。ここで、各パラメータは適切に定める必要がある。Q 関数の学習開始時刻を定める A は 1000 程度、キューの容量については実行環境ごとに設定せよ。またターゲットネットワークの更新頻度を定める B については 10 から 10000 などレンジが広いため注意が必要である。 ϵ は初期段階では 1 に近い値を、その後徐々に 0 に近づけていくといった工夫が考えられる。N については 2 から 10 程度のうちから任意の整数に設定して良い。横軸を環境におけるステップ回数、縦軸を累積報酬の移動平均として、その学習曲線をプロットせよ。

4.5 さらに進んで

以下のいずれかについて実装し、考察せよ。

- 問題設定において、
 $N - 1$ 個の交換操作の行動に加え、自らタスクを終了する行動を定義し、終了時点で正しくソートが出来たら報酬として 1 を、出来ていなかったら 0 を与える
 という環境を再度定義し、同様の実験を行い学習曲線をプロットし、その差異について考察せよ。
- これまでは入力として数字を用いてきたが、代わりに MNIST 画像を用いて同様の実験を行い、考察せよ。